本章路線

這章要回答的核心問題是：在任意交錯下，如何保證共享狀態正確，同時保留進度與等待上限？

前置知識。 process 與 thread 狀態、共享記憶體、交錯與原子操作。

學習目標

- 證明 mutual exclusion、progress 與 bounded waiting
- 用 semaphore 建立明確順序與背壓
- 辨識 deadlock、starvation、priority inversion 與 lost wake-up

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

競態條件與共享資料

多個執行流同時存取共享資料時，若至少一方會修改資料，最終結果可能取決於操作的交錯順序。這種時序造成的不確定性稱為 race condition（競態條件）。判斷程式是否正確，要先說清楚共享狀態應維持的不變量，以及哪些操作必須協調。

例如 producer 對 `counter` 加一，consumer 對它減一；若兩次操作都完整執行，且期間沒有其他修改，期望值應與初值相同。但一個「加一」或「減一」可能分成讀取、計算、寫回：兩者都讀到舊值時，較晚的寫回可能覆蓋較早的結果。

以下範例使用抽象機器：每一步讀取或寫回都可單獨觀察，同一執行流的步驟保持先後順序。這不代表 C/C++ 中未同步的非原子共享操作有固定結果；若構成 data race，其行為未定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

交錯執行：為何 counter 變成 4？

初始 `counter=5`。Producer 要加一，consumer 要減一；兩者各自先讀取、在暫存器計算，最後寫回。

時間	Producer	Consumer	counter	說明
T0	`R1=5`	—	5	producer 讀到 5
T1	`R1=6`	—	5	加一的結果尚未寫回
T2	—	`R2=5`	5	consumer 也讀到舊值
T3	—	`R2=4`	5	減一的結果尚未寫回
T4	`counter=6`	—	6	producer 寫回
T5	—	`counter=4`	4	consumer 覆蓋了 producer 的更新

若交換最後兩次寫回，結果是 6；若其中一個完整操作在另一個開始前完成，則結果是 5。此例的問題不是某一步算錯，而是兩個複合操作沒有受到共同同步協定保護。

何時需要同步

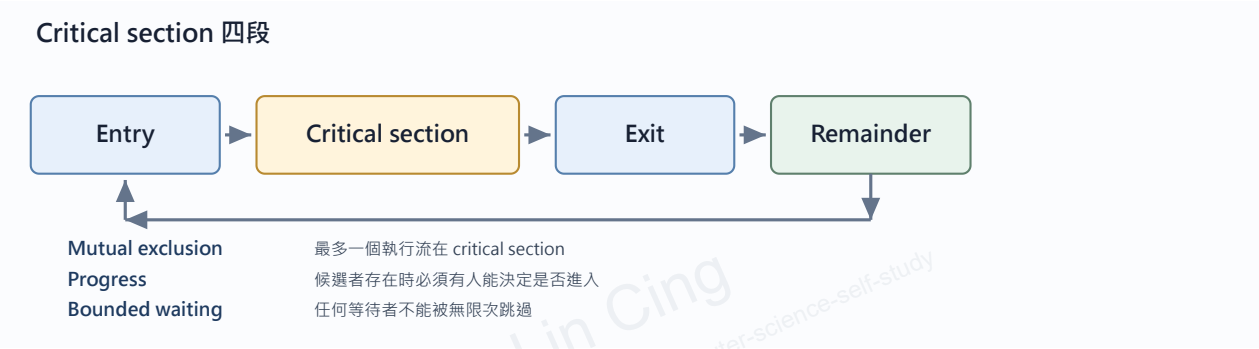
分析共享狀態時，可依序問四個問題：

1. 哪些可變位置會被多個執行流存取？是否有衝突的讀寫？
2. 抽象狀態要維持什麼不變量？例如 `0 ≤ size ≤ capacity`，或「有 writer 時不能有 reader」。
3. 哪些操作需要多個步驟，期間可能暫時破壞不變量？
4. 哪些觀察者不能看見中間狀態？使用什麼鎖或其他協定保證互斥、順序與資料可見性？

單一原子操作只保證它自己的不可分割性。例如更新鏈結串列時，即使修改 `next` 與 `size` 的兩個 store 各自原子，另一執行流仍可能在兩次 store 之間觀察到不一致。保護多欄位關係需要涵蓋所有相關存取的協定。

三個要求的精確界線

每個執行流反覆經過 entry section、critical section、exit section 和 remainder section。進入協定的三項要求各自回答不同問題：



要求	意義	違反時可能發生
Mutual exclusion（互斥）	同一時刻至多一個執行流在受保護的 critical section	兩個操作同時破壞共享不變量
Progress（進展）	臨界區空閒且有人要進入時，只由相關候選者參與決定，不能無限期延後選擇	臨界區空著，候選者卻一直無法決定誰進入
Bounded waiting（有界等待）	某執行流提出進入請求後，在其獲准之前，其他執行流能進入的次數有有限上界	系統持續工作，但特定等待者一直被插隊

三個要求的精確界線 （續）

互斥是安全性；進展與有界等待涉及活性及公平。證明活性時也要交代執行流會繼續被排程、已進入的臨界區會結束等假設；「其他人最多進入有限次」本身不是等待時間的毫秒上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三個彼此獨立的反例

互斥成立，但沒有 progress。 兩人使用嚴格輪替的 `turn`，每人只能在輪到自己時進入。若輪到 P1，但 P1 在 remainder section 不想進入，P0 即使想進入且臨界區空著，也只能等待 P1 改變 `turn`。

互斥與整體 progress 成立，但缺乏 bounded waiting。 不公平的鎖每次空出後總能讓某個競爭者取得，但同一位等待者可能被無限次越過。系統一直有進展，該等待者仍可能飢餓（starvation）。

沒有互斥。 entry section 不做任何檢查，所有候選者可立即進入；等待很短，卻可能同時修改共享資料，安全性立即失效。

這些例子說明，看到「有人能進入」不能當作每個人都有等待上界的證明；看到「沒有人等待」也不能代替互斥證明。

進入與離開協定的檢查

對進入協定 `enter_i` 和離開協定 `leave_i`，依序檢查：

1. 對所有允許的相對速度與交錯，是否都不會有兩個執行流同時位於同一受保護的臨界區？
2. 臨界區空閒且存在候選者時，選擇是否能完成，而不是由不想進入的執行流阻擋？
3. 某個請求提出後，其他執行流能在它之前進入幾次？是否存在有限上界？
4. 協定依賴哪些原子操作、記憶體順序、排程與持鎖者完成假設？錯誤或取消路徑是否仍符合契約？

有限測試可以找到反例，不能枚舉任意時間與所有交錯。安全性通常要對所有可達狀態證明；活性與公平需另行列出必要的環境假設。

Peterson 演算法的完整協議

Peterson 演算法只處理兩個執行流。令 `j=1-i`，共享 `flag[0]=flag[1]=false` 與變數 `turn`。此處先假設共享讀寫原子且具有 sequential consistency（順序一致性）。

```
P_i:
  flag[i] = true
  turn = j
  while (flag[j] && turn == j) { /* 等待 */ }
  critical_section_i
  flag[i] = false
  remainder_i
```

`flag[i]` 表示 Pi 想進入或仍位於臨界區；`turn` 在兩者都想進入時決定哪一方先等待。它不是鎖的持有者欄位：若另一方沒有競爭，某執行流即使最後寫了 `turn`，也可能直接進入。

互斥性證明

假設所有 `flag` 與 `turn` 的存取都原子且順序一致，反設 P0、P1 這兩次臨界區重疊。設 W0 是 P0 本輪寫入 `turn=1`，W1 是 P1 本輪寫入 `turn=0`；順序一致性保證兩者有先後。

若 W0 在 W1 之前，P0 的 `flag[0]=true` 也已在 W1 之前發生。當 P1 通過 while 時，P0 尚未離開本輪臨界區，因此 `flag[0]` 仍為 true；最後的本輪 `turn` 寫入為 W1，值為 0。P1 的等待條件 `flag[0] && turn==0` 成立，P1 不可能通過，矛盾。

若 W1 在 W0 之前，對稱地可得 P0 的等待條件成立，同樣矛盾。因此兩個臨界區不可能重疊。證明所比較的是兩次重疊嘗試中的寫入；進入臨界區後，該執行流直到離場才清除自己的 flag。

進展、有界等待與記憶體順序

Progress。 若只有 Pi 想進入，對方 flag 為 false，Pi 不必等待。若兩人都想進入，在順序一致的模型中 `turn` 為 0 或 1，等待條件不會同時擋住兩人；已在臨界區者離開時清除自己的 flag。

Bounded waiting。 從 Pi 完成 `flag[i]=true; turn=j` 起算，Pj 至多再進入一次。Pj 離開後清除 flag；若它再次競爭，必須先把 `turn` 設為 i，讓仍在等待的 Pi 不再因 `turn` 而受阻。若只從設旗標起算，Pi 可能尚未寫 turn，而 Pj 可能在旗標寫入之前已通過檢查；這時不能直接套用一次的界線。要推論 Pi 最終執行，仍需它和已進入者得到執行機會、臨界區能結束。

記憶體模型。 上述論證依賴順序一致的共享原子存取；在 C/C++ 中使用未同步的普通共享變數可能構成 data race，不能直接套用證明。可使用各相應位置的原子操作與 `memory_order_seq_cst`，或證明其他實作提供等價的順序與可見性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bakery 演算法：同號也能決定先後

有 n 個固定參與者，每人最多同時提出一個請求。`choosing[i]` 初始為 false，`number[i]` 初始為 0；0 表示尚未排隊。此處為便於證明，所有共享變數讀寫均假設為 sequentially consistent 的原子操作，票號不溢位。這是較強記憶體假設的教學版本；Lamport 原始論文討論的讀寫模型更弱。

```
choosing[i] = true
number[i] = 1 + max(number[0..n-1])
choosing[i] = false
for each j != i:
  while choosing[j]: wait
  while number[j] != 0 and (number[j],j) < (number[i],i): wait
critical section
number[i] = 0
```

配對使用字典順序：先比票號，票號相同再比身分。掃描 max 不是一個不可分割的快照：兩人可以同時看到最大值 0，兩人都選 1。若 P0、P1 都完成取號且分別為 (1,0)、(1,1)，P0 排在前面，P1 必須等待 P0 將自己的號碼清為 0。

Bakery：為何需要 choosing？

`choosing[j]` 表示 Pj 正在選號；觀察者先等它結束，才能依它已發布的號碼做比較。若省略這一步，初始全為 0 時可能出現：



修正：P1 先等 choosing[0] 結束，再比較票號

正確協議中 P1 會因 `choosing[0]=true` 停住，等 P0 發布結果後發現 (1,0) 優先。互斥證明依每人檢查對方的選號完成與票號順序，不能只用「取號必然不同」解釋。

當 Pi 完成取號後，已有或正在選號的其他請求至多 n−1 個；各參與者重新請求時，若 Pi 的號碼仍有效，新號碼須排在 Pi 之後。要推得 Pi 最終進入，還要假設選號／臨界區最終完成、相關執行緒持續獲得排程；這不是固定時間上限。

Bakery：一般 n 人的互斥證明

反設 Pi 和 Pj 的兩次請求同時在臨界區。Pi 檢查 Pj 時，Pi 的號碼已發布，直到 Pi 離開臨界區前都不變。

若 Pj 在 Pi 檢查它時已發布本輪非零票號，Pi 等到 `choosing[j]=false` 才比較。要讓 Pi 通過，必須 `(number[i],i) < (number[j],j)`。若 Pj 尚未完成對 Pi 的檢查，嚴格全序使 Pj 必須等待。若 Pj 早已檢查並忽略 Pi 的本輪請求，則它的選號已發布且會維持到離開；Pi 後來選號時會讀到 Pj 的號碼，選出更大的值，反而不能通過。若兩人選號交疊，`choosing` 會讓檢查等待選號完成，再以配對次序判定。

另一種時序是 Pi 讀到 `choosing[j]=false` 後，Pj 才開始選號，而 Pi 接著讀到 `number[j]=0`。兩次讀取並非原子快照，這種交錯可以發生。不過 Pj 是在 Pi 完成取號後才開始；若兩人臨界區重疊，Pj 掃描時 Pi 的非零票號仍有效，故 Pj 的新號碼嚴格較大，必須等待 Pi 清號。若 Pi 讀取 `choosing[j]` 時 Pj 已在選號，Pi 會等該次選號結束；旗標不保證 Pj 在 Pi 檢查之後不會開始新請求。

兩個情況都與同時進入矛盾，因此在所列 SC、無溢位和單一未完成請求假設下，任意 n 個參與者滿足互斥。

test-and-set 與 compare-and-swap

`test_and_set(L)` 原子地讀取 L 的舊值，並將 L 設成 true；`compare_and_swap(L,0,1)` 則只有在舊值為 0 時才寫入 1，並回傳舊值。下列是以 test-and-set 表示的抽象 spinlock：

```
acquire(L):
  repeat old = test_and_set(L) until old == false

release(L):
  L = false
```

假設 L 初值為 false、只有成功將 false 轉為 true 的執行流取得鎖，而且每次只有持有者在完成臨界區後釋放：第一次操作可讀到 false；在它釋放前，其他原子 test-and-set 只能讀到 true。因此同時最多一個持有者。

這個互斥推理不自動提供資料可見性：實際程式中的成功 acquire 與 release 寫入還必須具有適當的記憶體順序，才能讓下一個持有者看見受保護資料。

spinlock 的邊界

在 spinlock 中，等待者反覆嘗試，不主動睡眠。若臨界區極短、持鎖者能在另一顆核心繼續執行，這可能省下睡眠與喚醒的成本；長時間等待、單核心或持鎖者無法執行時，則可能浪費 CPU，甚至阻礙持鎖者前進。

單核心上的典型危險：程式持有鎖時被中斷；中斷處理常式又自旋等待同一把鎖。被中斷的持鎖者要等中斷返回後才可執行，處理常式卻等它釋鎖，因而無法前進。中斷環境需依系統規則處理，例如使用適當的中斷遮罩與非睡眠同步原語。

簡單 test-and-set 鎖只說明誰能成功取得，不保證等待者的 bounded waiting；特定等待者可能一直競爭失敗。要保證公平性，需另外設計及證明排隊／交接規則。

原子操作與 acquire／release 的區別

原子讀改寫保證該操作不被觀察為一半；鎖還需處理它保護的其他資料何時可見，以及編譯器或處理器可否重新排序。

同一把鎖上，前一持有者先更新共享資料、再執行 release；下一持有者成功執行與之配對的 acquire 後，才能依語言與 API 的同步契約讀取那些更新。所有共享存取都必須遵守相同協定；普通共享旗標不能取代原子與記憶體順序。

CAS 單獨只能提供指定位置的原子比較與更新。若用於 lock-free 結構，還要另行處理 ABA、節點生命週期和記憶體回收，以及系統與個別操作的 progress 保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mutex 的使用契約

Mutex 提供互斥及其 API 定義的同步契約。凡是由同一不變量約束的共享存取，都必須遵守對應的鎖規則：

```
acquire(m)
  更新或讀取受 m 保護的共享狀態
release(m)
```

具所有權語意的 mutex 通常要求由持有者解鎖；semaphore 的 signal 通常不要求呼叫者是先前 wait 的執行流。不同 API 的錯誤檢查與限制須依其契約確認，不能只憑名稱假設。

等待 mutex 的實作可以阻塞、短暫自旋或混合兩者；被喚醒只表示有機會繼續競爭，是否能立即取得鎖取決於實作契約。例外、錯誤和取消路徑也必須維持正確的所有權及釋放次數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

依固定順序取得鎖

為所有可能同時持有的鎖指定嚴格全域順序，例如 `ord(A)=1`、`ord(B)=2`；每個執行流只在已持有的所有鎖都小於新鎖時，才可要求新鎖。

假設仍出現等待環。對環上的每個執行流，它持有某把鎖、正等待另一把，因此等待的鎖順序都必須比它所持有的鎖大。沿整個環走一圈便得到 $\mathrm{ord}(L_1) < \mathrm{ord}(L_2) < \dots < \mathrm{ord}(L_1)$，與嚴格順序不可能小於自身矛盾；因此遵循此規則的鎖等待邊不能形成環。

此結論依賴所有相關路徑都遵循同一順序；callback、鎖升級、例外與取消路徑也不能例外。動態物件可用穩定且唯一的 key 排序後取得鎖；只按轉帳的 from／to 參數順序取得兩把鎖，仍可能出現相反順序。本論證排除這類鎖循環，不能因此保證排程公平或排除其他資源形成的死結。

Semaphore 的兩種實作

資源型 semaphore 以 permit 表示可使用的資源。抽象的 binary semaphore 最多保有一個可用 permit；另一種帶號誌計數器的 blocking 實作可用負數表示等待者，內部整數不能直接當成抽象資源數。

Busy-wait 版本的關鍵是「確認有 permit 並扣減」必須是**同一個原子動作**。以下以原子 CAS 描述概念，假設 S 的所有操作均有正確的原子性與可見性；資源型 signal 也不得超出約定容量：

```
wait(S):
  loop:
    old = atomic_load(S)
    if old > 0 and CAS(S, old, old-1) succeeds: return
signal(S): atomic_fetch_add(S, 1)
```

若只寫 `while(S<=0) {}; S--` 而沒有原子檢查及扣減，S＝1 時兩個執行緒都可能先看到 1，然後都通過；只有一個 permit，卻產生兩個持有者。真實實作還須選擇相應的記憶體順序與處理上限。

阻塞式 semaphore：等待者與 permit 交接

下列是特定的 signed-counter 教學模型：更新計數、登記等待者及提交睡眠須與 signal 原子協調；被選中的等待者取得保留的 permit，即使尚未排到 CPU，新來者也不能取走該 permit。

```
wait(S):
  原子地 S--；若 S < 0，登記並阻塞
signal(S):
  原子地 S++；若 S <= 0，選出等待者並交付 permit
```

容量 1 的例子：T0 `wait` 使 S：1→0，取得 permit；T1 `wait` 使 0→−1 並等待；T0 `signal` 使 −1→0，選中 T1 並保留 permit 給它。T1 恢復時從原 wait 返回，不再扣一次；它之後 `signal` 才使 0→1，留下可用 permit。

穩定邊界下可用 permit 為 max(S,0)，佇列中等待者為 max(−S,0)；已交接但尚未執行者的 permit 已分配。這是特定實作，不是所有 POSIX semaphore 的內部表示。資源型 semaphore 可用「空閒＋已分配＝容量」檢查；事件型 semaphore 不一定有固定容量。

有界緩衝區的操作順序

令 `empty=3`、`full=0`、`mutex=1`，buffer 為空。下列是合法事件的前後狀態。

已完成的操作	empty	full	buffer	可觀察事實
初始	3	0	`[]`	三個空槽
put A	2	1	`[A]`	一項已發布
put B	1	2	`[A,B]`	`empty+full=3`
put C	0	3	`[A,B,C]`	下次 put 必須等待
get A	1	2	`[B,C]`	取出後釋放容量
put D	0	3	`[B,C,D]`	容量仍為 3

producer 應 `wait(empty); wait(mutex); ...; signal(mutex); signal(full)`。consumer 對稱地先 `wait(full)`。先取得 mutex 再等資源可能持鎖睡眠，令能產生該資源的對方也卡在 mutex。

死結、飢餓與 priority inversion

設 S、Q 初值皆為 1。P0 先取得 S、P1 先取得 Q；若 P0 接著等 Q、P1 等 S，兩者各自持有對方需要的資源，形成 deadlock。相反地，系統仍不斷完成工作，但特定等待者一直被跳過，是 starvation。

低優先權 L 持有高優先權 H 所需的鎖時，若中優先權 M 不斷搶占 L，H 會間接等待 M，這是 priority inversion。Priority inheritance 可暫時提高 L 的有效優先權，讓它有機會完成並釋鎖。

單鎖且沒有其他 donation 的簡化例子中，釋鎖後 L 可恢復原優先權；若它仍持有其他鎖、存在其他高優先權等待者或鏈式繼承，必須依剩餘等待關係重新計算，不能一律恢復原值。繼承改善特定優先權反轉，仍不等於一般的公平或無死結保證。

有界緩衝區的正確性條件

容量為 n 的緩衝區使用三個不同的同步物件：`empty` 限制可用空槽，`full` 限制可取出的項目，`mutex` 則保護槽位、`head`、`tail` 與 `size` 的複合更新。安全條件包括 `0 ≤ items ≤ n`；producer 先等待 `empty` 才取得 mutex，consumer 對稱地先等待 `full`。若 producer 持有 mutex 卻等待空槽，consumer 可能因無法取得 mutex 而不能取出資料，形成等待環。

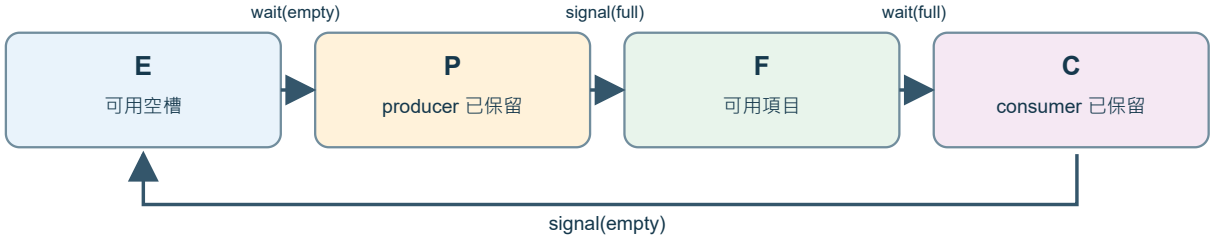
一般記帳必須計入已授權但尚未完成的操作。令 E、F 是尚未保留的 empty、full 可用 permit；P 是已取得 empty permit、尚未完成 `signal(full)` 的 producer reservation 數；C 是已取得 full permit、尚未完成 `signal(empty)` 的 consumer reservation 數。直接交給已喚醒但尚未執行者的 permit，自授權時就算入 P 或 C。

在正常執行且每次保留都會依協定完成的模型中，各原子協定轉移邊界維持 `E + F + P + C = n`。Producer 成功等待將 E 轉成 P；發布 full 時，P 轉成 F，或直接交給等待中的 consumer 而轉成 C。Consumer 對稱地將 F 轉成 C，釋出 empty 時再轉成 E 或直接轉成 P。寫入或取出資料本身不改變這項 permit 分類。

例如 n＝3，初始 `(E,F,P,C)=(3,0,0,0)`；一個 producer 剛取得 empty permit、尚未寫入時為 `(2,0,1,0)`，此時 `E+F=2`，不是 3。只有沒有在途 reservation 的靜止邊界才有 `E+F=n`。E、F 是非負可用 permit，不能直接拿可能含等待者的 signed semaphore 原始計數代入；F 在任意時刻也不一定等於 items。若加入取消或失敗，還須設計 reservation 的回復規則。

在途保留如何維持容量守恆

容量 3：permit 在四種狀態間流轉



直接交棒：P 可直接轉 C；C 可直接轉 P

事件	E	F	P	C	說明
空緩衝區	3	0	0	0	三個空槽尚未保留
producer 成功 wait(empty)	2	0	1	0	空槽已保留，尚未發布項目
producer 完成寫入並 signal(full)	2	1	0	0	一個項目可供取得
consumer 成功 wait(full)	2	0	0	1	該項目已保留，尚未釋放空槽
consumer 取出並 signal(empty)	3	0	0	0	回到空緩衝區

每一列 E＋F＋P＋C 均為 3。圖中箭頭描述 permit 的分類，而非每個資料複製步驟；直接交棒時可由 P 直接轉 C，或由 C 直接轉 P。取消／故障路徑須另外回復或完成已保留的授權。

讀者與寫入者的公平策略

策略	新 reader 遇到等待中的 writer	可能長期等待者
Reader-preference	writer 尚未取得資源時，新 reader 仍可加入	writer
Writer-preference	writer 開始等待後，新 reader 不再開始	reader

Reader-preference 的典型協議用 `read_count` 記錄目前讀者數，並以同一把 mutex 保護計數更新。第一個 reader 以 wait 取得保護共享資料的 `rw_sem`；最後一個 reader 以 signal 釋放它，允許 writer 進入。

第一位與最後一位 reader 可能不是同一個執行緒，所以這裡的 `rw_sem` 必須是允許跨執行緒 signal 的 binary semaphore 等無 owner permit；不能直接把它換成只允許持有者解鎖的 POSIX mutex。保護 `read_count` 的 mutex 則由取得它的 reader 自己釋放。

哲學家進餐：互斥與死結

五位哲學家共用五支筷子，每人需要左、右兩支才能進食。若一律先拿左筷，再拿右筷：

```
wait(chopstick[i]);
wait(chopstick[(i+1)%5]);
eat;
signal(chopstick[(i+1)%5]);
signal(chopstick[i]);
```

可能的交錯是：五人各拿到自己的左筷，再各自等待已被鄰居持有的右筷。每個人都等待另一人先釋放，形成五節循環等待；每支筷子雖然仍保持互斥，系統卻無法前進。

常見設計包括：限制最多四人同時進入取筷協定；用受保護的協定只在兩支筷子均可取得時才授權進食；或者讓奇偶哲學家使用相反取得順序，破壞這個環形等待。以第一種方法為例，若四人各持左筷，仍有一支筷子未被拿作左筷；相鄰的一位可取得第二支並完成，前提是可執行者最終被排程、進食後會釋放。

排除這個循環等待不等於保證每人都有有限等待時間。是否會飢餓仍取決於等待者選擇、排程與反覆競爭的公平性。

等待的是條件，不是一張通知票

採 Mesa 語意；共享佇列 `q` 初始為空，P 只放入一筆 A，C1、C2 各呼叫一次 `get()`。佇列的所有修改和 `q.empty()` 檢查均持有同一把 mutex `m`，`not_empty` 是 condition variable。

```
get():                          put_A():
  lock(m)                         lock(m)
  while q.empty():                q.push_back(A)
    wait(not_empty, m)            signal(not_empty)
  x = q.pop_front()               unlock(m)
  unlock(m)
  return x
```

`wait` 相對於其他執行緒取得 m 並通知的行為，原子地完成釋鎖與進入等待的交接；成功返回前必須重新取得 m。`signal` 不會立即把鎖交給等待者，也不為它保留 A。沒有等待者時，通知本身不會累積成可供將來取用的 permit。

被通知，不代表下一個拿到鎖

時間向下；**P** 只放入一個 **A**；本例沒有額外喚醒

	Consumer C1	Producer P	Consumer C2	mutex / queue
T0	wait：釋鎖 + 入 CV			m=—, q=[]
T1		lock；放入 A		m=P, q=[A]
T2	被通知；尚未持鎖	signal C1；仍持鎖		m=P, q=[A]
T3		unlock →	lock；取 A；unlock	m=—, q=[]
T4	重取鎖；發現 q 空			m=C1, q=[]
T5	while → 再 wait			m=—, q=[]

依圖追蹤：C1 先在空佇列上等待；P 放入 A 並通知 C1，但仍持有 m。T3 依序是 P 釋鎖、C2 先取得 m 並取走 A，然後 C2 釋鎖；此例不假設 mutex 依 FIFO 選取等待者。

C1 在 T4 從 wait 返回時確實重新取得 m，但佇列已再次變空。這不是 spurious wakeup：若只用 `if` 檢查一次，C1 會對空佇列執行 `pop`；使用 `while` 則會重新檢查並在 T5 繼續等待。

把三種狀態分開記錄

下表中的 CV 是 condition variable 的等待佇列；E 是已收到通知、但尚未重新取得 `m` 的執行緒集合，並不是核心 ready queue。表格依序記錄每個事件完成後的狀態。

事件	m 的持有者	佇列 q	CV	E
T0：C1 進入 wait	無	`[]`	`{C1}`	`{}`
T1：P 放入 A	P	`[A]`	`{C1}`	`{}`
T2：P 通知 C1	P	`[A]`	`{}`	`{C1}`
T3：C2 取走 A 並釋鎖	無	`[]`	`{}`	`{C1}`
T4：C1 重新取得 m	C1	`[]`	`{}`	`{}`
T5：C1 再次 wait	無	`[]`	`{C1}`	`{}`

T2 的通知改變 C1 的等待資格，但不保留佇列中的 A；是否取得 mutex 及何時執行仍是另外的狀態。

遺失喚醒：等待之前的空窗

錯誤：自行拆開 unlock 與等待

1. C1：檢查空，unlock；尚未入 CV
2. P：放 A、signal；沒有等待者
3. C1：現在才入 CV，睡著；q=[A]

有資料卻睡著；通知不會累積。

正確：wait 的原子契約

1. C1：檢查空，wait 釋鎖並登記
2. P：取得鎖，放 A、signal，再 unlock
3. C1：重取鎖，重檢非空，取走 A

P 先取得鎖時，C1 看到 A 就不 wait。

左圖是假設某個錯誤的等待實作先解鎖，再另外登記並阻塞；它不是合法的 `pthread_cond_wait` 實作契約。P 可能在這個空窗更新條件並通知，但當時 CV 還沒有等待者；C1 隨後才睡著，若再無事件就可能永久等待。

正確的 condition wait 以原子交接排除此空窗。若 P 先完成，且 A 尚未被其他 consumer 取走，C1 在持有 m 時看到佇列非空，就不等待；若 A 已被取走，C1 看到空佇列並繼續等待是正確行為。通知沒有累積成獨立的「票」，所以等待者始終應依共享條件判斷。

分開證明安全性與活性

安全性：每次 pop 前都在持有 m 的情況下確認 `!q.empty()`；從檢查到取值之間，其他執行緒不能取得 m 改變 q。Wait 成功返回只保證重新取得 mutex，並不保證條件仍成立。

活性：`while` 防止錯誤取值，卻不保證每位 consumer 都能取得資料。本例只有一筆 A，若 C2 已取走，C1 繼續等待是合理結果。要證明某個等待者最終完成，還需條件再次成立、相應通知、排程與鎖取得等假設；不能只由 `while` 推出。

若 broadcast 喚醒兩位 consumer，但只有一筆 A，先取得 m 的一人取走 A；另一人仍須在重新取得 m 後檢查條件並等待。通知更多人不會增加資料，也不自動建立有限等待上界。

Hoare 與 Mesa 如何交棒

前述反例使用 Mesa：通知者繼續持有鎖並執行，等待者之後才重新取得鎖。不同 monitor 的交接契約不同，不能把某一模型的條件保證直接套用到另一模型。

交接方式	signal 後的執行權與鎖	等待者如何判讀
Hoare signal-and-wait	立即交給被通知者，通知者等待	只有在 signal 前已建立目標條件，且模型排除虛假喚醒、逾時等情況時，才能利用交接點的條件保證
Mesa signal-and-continue	通知者繼續執行並持鎖，等待者之後重取	其他執行緒可能先改變共享狀態，因此必須重檢 predicate
Concurrent Pascal signal-and-exit	通知者離開 monitor 後再交棒	依該語言定義的交接與條件語意判讀

固定版本的同步機制

以下以原書第 9 版討論的 Windows 7、Linux 2.6 和 Solaris 案例為歷史參考。各系統的版本、核心設定與呼叫情境會影響機制；不能把一個案例的內部實作當成所有版本的 API 契約。

案例	同步工具	使用時需檢查的條件
Windows 核心	spinlock 保護短時間共用資料	`KeAcquireSpinLock` 要求呼叫時 IRQL ≤ DISPATCH_LEVEL，將目前處理器的 IRQL 提高到 DISPATCH_LEVEL；配對釋放時恢復舊值
Windows 使用者模式	mutex、semaphore、event 等等待物件；另有 process 內 critical section	等待物件與 critical section 的所有權、可見範圍和喚醒條件不同
Linux 教材案例	atomic 操作、spinlock、mutex 與 semaphore	是否可睡眠，以及是否須額外處理本機中斷或核心搶占，取決於鎖、設定與執行環境
Solaris 教材案例	adaptive mutex、reader-writer lock、turnstile	持鎖者正在其他 CPU 執行時可考慮 spin；否則可睡眠；turnstile 可支援 priority inheritance
POSIX	mutex、condition variable、reader-writer lock 及 semaphore	檢查對應 API 的等待、錯誤、所有權與程序間共享契約

固定版本的同步機制 （續）

`KeAcquireSpinLock` 持鎖期間，一般執行緒排程不能搶占目前處理器上的持鎖路徑，但更高 IRQL 的硬體中斷仍可能發生。ISR 使用的 interrupt spinlock 和 DIRQL 有另一套呼叫規則，不能把 executive spinlock API 直接帶入所有中斷處理常式。持 spinlock 的區段也不能執行可能睡眠的等待。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何同名 primitive 不能直接互換

Windows 的 user-mode critical section 是同一 process 內的遞迴互斥工具；它不等於同名的核心程式碼區段。可設定 spin count：在多處理器競爭時先嘗試，再依 API 進入等待；單處理器的 spin count 會被忽略。等待執行緒取得所有權的順序不保證 FIFO，實作的等待物件也不應被概括成「必定配置 kernel mutex」。

Linux 教材中的一般 spinlock 用於不能睡眠的短區段；在某些單核心及核心設定下，不需真正跨核心自旋，但仍須處理核心搶占和可能的中斷競爭。`spin_lock`、`spin_lock_irqsave` 的中斷處理效果不同；不能把「停用 preemption」誤當成「所有中斷都被遮罩」。較新 Linux 的 PREEMPT_RT 對 `spinlock_t` 有不同實作與規則，須另按目標版本確認。

比較替代方案時，至少釐清：呼叫發生在 user／kernel／interrupt context；等待期間能否睡眠；鎖是否有 owner、能否跨 process；是否允許重入、取消或跨執行緒釋放；以及 API 對可見性、喚醒次序與錯誤的保證。名稱相似不能替代這些契約。

等待時間與公平。 支援 timeout 只讓呼叫者在條件未滿足時可走逾時路徑，不代表它在期限內必然取得鎖，也不建立 FIFO 或 bounded waiting。實際返回時間還受排程與時鐘等 API 條件影響。

交易記憶體

交易記憶體以 transaction 表示一組共享讀寫：在實作承諾的隔離範圍內，成功 commit 時整組效果生效；abort 時交易記憶體寫入不生效，之後可重試。以下是概念偽碼，`atomic` 不是可直接貼入 C++ 的通用語法。

```
atomic transaction:
  x = x - 25
  y = y + 25
```

例如初始 `x=100, y=40`，成功 commit 後為 `(75,65)`，總和仍為 140；若交易 abort，則這次交易未提交的記憶體更新不生效。在所選 transaction 系統提供的隔離契約下，另一個遵守同一協定的觀察者不會把只減 x、未加 y 的中間狀態當作已提交結果。

並行衝突可能觸發 abort；容量限制、排程或實作限制也可能導致失敗。重試次數與進度須另作設計，交易不保證公平或每次最終成功。I/O、網路傳送等不可逆外部效果通常不能靠回滾記憶體撤銷，必須移出可重試區段或採用明確的補償／提交協定。

STM 通常由 compiler／runtime 記錄並驗證讀寫；HTM 依硬體機制追蹤衝突與資源限制。若加入鎖作 fallback，交易路徑和鎖路徑必須共同遵守同一互斥協定；不能假設不參與交易的普通讀寫自動受到保護。交易式編程可避免某些手寫鎖順序問題，但仍可能反覆失敗或飢餓。

OpenMP、immutability 與實際邊界

OpenMP `#pragma omp critical(name)` 保護其結構化區塊：在同一 contention group 內，相同名稱的 critical 區段一次最多一個 thread 執行，不因 thread 所屬 team 不同而改變。未命名的 critical 區段視為具有同一個未指定名稱；不同名稱不提供彼此之間的互斥。

例如兩個區段都更新 `total`，若分別使用 `critical(A)` 與 `critical(B)`，不能靠這兩個不同名稱保證彼此互斥。所有會衝突的存取必須遵守相同同步協定；單獨保護其中一個寫入點並不足夠。

Immutable value 建立後不再原地修改，因此多個執行流可在有效生命週期內共享同一個舊值；但建立新版本、發布給讀者、物件回收以及檔案、網路或 mutable database 仍可能需要同步。使用 immutable 並不自動保證整個程式沒有競爭。

Message passing 可透過複製或依協定轉移所有權，減少直接共享可變資料；仍須明定通道容量、順序、backpressure、取消和失敗重送。比較方案時也要計入複製、分配與等待成本，而不是只比較語法。

如何選擇同步工具

問題	必要資訊	應檢查
只有一個 critical section	臨界區長度、是否可 sleep、是否有 owner	lock acquire/release 是否成對
有固定數量 resource	capacity、初始 permits、誰可 signal	semaphore 是否可能超額 signal
等待 predicate	predicate 變數、通知條件、喚醒後重檢	CV 是否綁定同一 mutex
多個 locks	全域順序、動態 key、取消路徑	每條路徑是否仍依序取得
多 readers 與 writers	公平版本、writer 是否可等待	是否出現 writer 或 reader starvation

完成選型後，逐事件列出 mode、owned resources、waiting set 與 predicate。只在平均測試中看到正確答案，不能排除少見交錯；一般性證明與可重跑反例要分開提出。

本章重點

- critical section 三性質分別對應安全、活性與公平
- semaphore 與 monitor 的錯誤多在等待條件而非 API 語法
- 經典假設不必然適用弱記憶體或現代硬體

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 5. Lamport, A New Solution of Dijkstra’s Concurrent Programming Problem, 1974. pthread_cond_wait(3p); Windows spin locks; Windows critical sections; Linux lock types; OpenMP critical.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study